UNIVERSIDAD AUTÓNOMA DE SINALOA

FACULTAD DE INFORMÁTICA CULIACÁN

MANUAL DE PRÁCTICAS DE MONGODB

COLECCIÓN, CONSULTA, ACTUALIZACIÓN Y ELIMINACIÓN DE DOCUMENTOS CON MONGODB



ASIGNATURA: Bases de Datos Emergentes

PROGRAMA EDUCATIVO: Licenciatura en Ingeniería en Ciencia de Datos

RESPONSABLE: Dra. Lidia Yadira Pérez Aguilar

FECHA: agosto 2026

DATOS • CONOCIMIENTO • DECISIONES

Tecnología que transforma información en valor

PRÁCTICA 5. Modelado documental: embedding y referencing

Objetivo

Comprender y aplicar las estrategias de embedding y referencing en MongoDB mediante el modelado de información relacionada y la creación de las colecciones clientes, proveedores y ventas, integrándolas con la colección productos para representar diferentes tipos de relaciones dentro de una base de datos documental.

Embedding: El embedding consiste en almacenar información relacionada dentro del mismo documento.

Ejemplo:

```
{
    cliente_id: "C001",
    nombre: "Ana López",
    direccion: {
        ciudad: "Culiacán",
        estado: "Sinaloa",
        cp: "80000"
    }
}
```

En este caso, `direccion` es un documento embebido dentro del cliente.

Referencing: El referencing consiste en almacenar los datos en documentos o colecciones diferentes y utilizar un identificador para establecer la relación.

Por ejemplo:

```
{
    venta_id: "V001",
    cliente_id: "C001"
}
```

`cliente_id` permite identificar al cliente relacionado con la venta.

MongoDB no obliga a utilizar claves foráneas como una base de datos relacional. La relación forma parte del diseño de la base documental.

Actividades

- Ejecutar el código en el orden indicado y verificar los resultados.
- Registrar evidencia de las consultas ejecutadas.
- Guardar el código en un documento .js
- Subirlo al repositorio GitHub

Entregable

- Coding completo y ejecutable.
- El un pdf con las respuestas a los ejercicios.

0. Verificación de la base de datos

- **Inicie MongoDB y posteriormente mongosh.**
- **Seleccione la base de datos utilizada en las prácticas anteriores**

```
use ventas_online
```

- **Compruebe las colecciones existentes:**

```
show collections
```

- **Contar los documentos**

```
db.productos.countDocuments()
```

Resultado esperado: 50 documentos.

1. Creación de nuevas colecciones

Cree las nuevas colecciones:

```
db.createCollection("clientes")
db.createCollection("proveedores")
db.createCollection("ventas")
```

Compruebe:

```
show collections
```

2. Embedding en clientes

Utilizar embedding para almacenar la dirección y los medios de contacto de un cliente.

Insertar el siguiente documento:

```
db.clientes.insertOne({
    cliente_id: "C001",
    nombre: "Ana López",
    correo: "ana.lopez@email.com",
    telefono: "6671234567",
    direccion: {
        calle: "Álvaro Obregón",
        numero: 120,
        ciudad: "Culiacán",
        estado: "Sinaloa",
        cp: "80000"
    },
    preferencias: ["tecnologia", "electronica"],
    activo: true
})
```

Consultar el documento:

```
db.clientes.findOne({cliente_id: "C001"})
```

Observar que `direccion` no se encuentra en otra colección. Está almacenada dentro del documento del cliente.

Prueba: Consultar únicamente el nombre y la ciudad:

```
db.clientes.find(
    {cliente_id: "C001"},
    {_id: 0, nombre: 1, "direccion.ciudad": 1}
)
```

3. Embedding con arrays

Insertar un segundo cliente:

```
db.clientes.insertOne({
    cliente_id: "C002",
    nombre: "Carlos Martínez",
    correo: "carlos@email.com",
    telefono: "6672345678",
    direccion: {
        ciudad: "Mazatlán",
        estado: "Sinaloa",
        cp: "82000"
    },
    preferencias: [
        "computacion",
        "accesorios",
        "audio"
    ],
    activo: true
})
```

Consultar los clientes interesados en accesorios:

```
db.clientes.find(
    {preferencias: "accesorios"},
    {_id: 0, cliente_id: 1, nombre: 1}
)
```

En este ejemplo tenemos dos tipos de estructuras embebidas:

```
direccion
  ↳subdocumento

preferencias
  ↳ array
```

4. Crear proveedores

Utilizar una colección independiente para almacenar proveedores.

```
db.proveedores.insertMany([
    {
        proveedor_id: "PR001",
        nombre: "Tecnología del Pacífico",
        ciudad: "Culiacán",
        estado: "Sinaloa",
        activo: true
    },
    {
        proveedor_id: "PR002",
        nombre: "Distribuidora Digital",
        ciudad: "Guadalajara",
        estado: "Jalisco",
        activo: true
    },
    {
        proveedor_id: "PR003",
        nombre: "Computación del Norte",
        ciudad: "Monterrey",
        estado: "Nuevo León",
        activo: true
    }
])
```

Comprobar:

```
db.proveedores.find()
```

y:

```
db.proveedores.countDocuments()
```

Resultado esperado: `3`

5. Referenciar un proveedor

Relacionar algunos productos existentes con proveedores. Primero consultar un producto para identificar un `sku` existente:

```
db.productos.findOne()
```

Suponer que existe:

```
P001
```

Actualizar el producto:

```
db.productos.updateOne(
    {sku: "P001"},
    {$set: {proveedor_id: "PR001"}}
)
```

Consultar:

```
db.productos.findOne(
    {sku: "P001"},
    {_id: 0, sku: 1, nombre: 1, proveedor_id: 1}
)
```

Aquí:

```
productos.proveedor_id
```



```
proveedores.proveedor_id
```

representa una referencia.

6. Crear una venta

Crear una venta utilizando embedding y referencing simultáneamente. Antes de hacerlo, consultar algunos productos existentes:

```
db.productos.find(
    {},
    {_id: 0, sku: 1, nombre: 1, precio: 1}
).limit(5)
```

Utilizar `sku` y precios que realmente existan en su colección.

Ejemplo:

```
db.ventas.insertOne({
    venta_id: "V001",

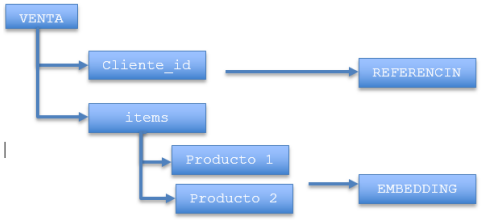
    cliente_id: "C001",

    fecha: ISODate("2026-09-01T10:30:00Z"),

    items: [
        {
            sku: "P001",
            cantidad: 1,
            precio: 1200
        },
        {
            sku: "P002",
            cantidad: 2,
            precio: 350
        }
    ],

    total: 1900,
    canal: "online",
    estado: "completada"
})
```

En este documento tenemos las dos estrategias:



7. Consultar información embebida

Mostrar ventas que contengan el producto `P001`:

```
db.ventas.find({
    "items.sku": "P001"
})
```

Mostrar únicamente el identificador de venta, cliente y productos:

```
db.ventas.find(
    {"items.sku": "P001"},
    {
        _id: 0,
        venta_id: 1,
        cliente_id: 1,
        "items.sku": 1
    }
)
```

8. Consultar una referencia

Consultar las ventas realizadas por el cliente `C001`:

```
db.ventas.find({
    cliente_id: "C001"
})
```

Ahora consultar al cliente:

```
db.clientes.findOne({
    cliente_id: "C001"
})
```

Observar que se estan realizando las dos consultas por separado.

9. Relación uno a muchos

Tenemos que un cliente puede tener muchas ventas. Por lo tanto, tenemos una relación conceptual uno a muchos.

Ejecutar:

```
db.ventas.insertOne({
    venta_id: "V002",
    cliente_id: "C001",
    fecha: ISODate("2026-09-03T16:15:00Z"),
    items: [
        {
            sku: "P003",
            cantidad: 1,
            precio: 800
        }
    ],
    total: 800,
    canal: "tienda",
    estado: "completada"
})
```

Ahora:

```
db.ventas.find(
    {cliente_id: "C001"},
    {_id: 0, venta_id: 1, cliente_id: 1, fecha: 1}
)
```

Ejercicios de la Práctica 5

Para estos ejercicios se requiere la captura de pantalla de los resultados obtenidos debajo de cada ejercicio, guárdala en PDF y nómbralo: Practica05-Modelado-Embedding-Referencing.pdf

1. Completar la colección clientes hasta obtener un total de 50 clientes (Los identificadores deberán ser únicos y consecutivos: C001 C002 C003 ... C050). Cada documento deberá contener como mínimo:

```
{
    cliente_id: "C003",
    nombre: "Nombre del cliente",
    correo: "correo@email.com",
    telefono: "6670000000",

    direccion: {
        ciudad: "Culiacán",
        estado: "Sinaloa",
        cp: "80000"
    },

    preferencias: [
        "tecnologia",
        "electronica"
    ],
```

```
    activo: true
}
```

2. Completar la colección proveedores hasta obtener 10 proveedores (Los identificadores deberán seguir la secuencia: PR001, PR002, ...PR010). Cada proveedor deberá contener:

```
{
    proveedor_id: "PR004",
    nombre: "Nombre del proveedor",
    ciudad: "Ciudad",
    estado: "Estado",
    activo: true
}
```

3. Los 50 productos deberán quedar asociados con alguno de los 10 proveedores. Para ello, agregar a cada producto: proveedor_id. Ejemplo:

```
db.productos.updateMany(
    {categoria: "Electronica"},
    {$set: {proveedor_id: "PR001"}}
)
```

4. Verificar las referencias de proveedores, Ordenar los resultados primero por proveedor_id y después por nombre. Muestre: sku, nombre, categoría, proveedor_id

5. Completar la colección ventas hasta obtener 100 ventas (Los identificadores deberán ser únicos: V001, V002, V003, ... V100). Cada venta deberá contener:

```
{
    venta_id: "V003",
    cliente_id: "C015",
    fecha: ISODate("2026-09-05T12:30:00Z"),
    items: [
        {
            sku: "P010",
            cantidad: 2,
            precio: 450
        }
    ],
    total: 900,
    canal: "online",
    estado: "completada"
}
```

Nota: Procurar que existan ventas con uno, dos y tres o más productos. Utilizar diferentes clientes, productos, fechas, canales y estados. Para `canal` utilizar valores como `online, tienda y app`; para `estado` utilizar `completada, pendiente y cancelada. Los cliente_id y sku` utilizados deberán existir previamente en sus respectivas colecciones.

6. Mostrar las primeras 10 ventas.
7. Seleccionar un cliente_id que tenga varias ventas y muestre todas sus operaciones.
8. Seleccione un sku existente y localice todas las ventas en las que aparezca ese producto.
9. Obtener todas las ventas realizadas mediante online
10. Obtener todas las ventas realizadas mediante app.
11. Obtener las ventas cuyo estado sea completada.
12. Obtener las ventas cuyo estado sea cancelada. Mostrar solo venta_id, cliente_id, fecha total estado

13. Mostrar las ventas ordenadas de mayor a menor según total. Mostrar solo venta_id, cliente_id, fecha, total
14. Obtener las 10 ventas con mayor importe total. Combinar find(), sort(), limit().
15. Mostrar las ventas en páginas de 10 documentos. Obtener Página 1, Página 2, Página 3.
16. Seleccionar una ciudad existente en los documentos y obtenga todos los clientes que residan en ella. Utilizar el campo: "direccion.ciudad".
17. Obtener los clientes que tengan una preferencia por tecnología. Mostrar: cliente_id, nombre, preferencias.
18. Seleccionar una venta que contenga al menos tres productos.
19. Seleccionar una venta y determine a qué cliente pertenece.
20. Mostrar las colecciones. Deben aparecer: productos, clientes, proveedores, ventas.
21. Mostrar el número de documentos para cada colección. El resultado debería ser:

- Productos 50
- Clientes 50
- Proveedores 10
- Ventas 100

Guardar el script al terminar la práctica

- Dentro de `mongosh`, escribe:

```
history()
```

Esto muestra el historial de comandos ejecutados durante la sesión.

- Después selecciona y copia el historial y pégalo en Visual Studio Code o Bloc de notas.
- Lo guárdalo como:

```
Practica05-Modelado-Embedding-Referencing.js
```

- Súbe a tu repositorio:
    - El archivo: Practica05-Modelado-Embedding-Referencing.js
    - El archivo: Practica05-Modelado-Embedding-Referencing.pdf (Con la portada)